



Final Year Project

Software Requirement Specification (SRS) Document

Project Members

Name	Roll Number
M. Sohaib Saleem (Lead)	2K22 BSCS 301
Abdul Wahab	2K22 BSCS 330
Muhammad Rehan	2K22 BSCS 339
Abdullah Khan	2K22 BSCS 342

Table of Contents

Software Requirements Specification (SRS)	1
Project: Intelligent, State-Aware Security and Monitoring System	1
1. Requirement Engineering Process	1
1.1 Requirement Elicitation	1
1.2 Requirement Analysis (Collection Analysis)	1
1.3 Requirement Specification	2
1.4 Requirement Validation	2
2. The Purpose of the Project	2
2.1 The User Business / Problem	2
2.2 Goals of the Project	2
3. The Stakeholders	2
4. Mandated Constraints	3
5. Scope of the Work (Use Case Diagram)	3
5.1 Diagram Description:	3
5.2 Diagram:	3
6. Functional Requirements	4
6.1 Mode Management	4
6.2 Motion Detection & AI Verification	4
6.3 Intrusion Handling & Recording	5
6.4 Alerts & Communication	5
7. Non-Functional Requirements	5
7.1 Usability	5
7.2 Performance	5
7.3 Reliability	5
8. Architectural Design	5
8.1 High-Level Architecture	6
8.2 Data Flow Diagram (DFD) Description	7
9. Future Scope (Project Evolution)	7
Appendix: A Glossary of Terms & Acronyms	7
A	7
C	8
F	8
G	8
I	9
J	9
L	9
M	9
N	10
P	10
R	10
S	10
T	11

Software Requirements Specification (SRS)

Project: Intelligent, State-Aware Security and Monitoring System

Version: 1.0

Status: Working

1. Requirement Engineering Process

Before defining the specifications, the following engineering steps were undertaken to ensure the project addresses the core problem effectively.

1.1 Requirement Elicitation

The elicitation process focused on identifying the shortcomings of current market solutions.

- **Market Gap:** Observation of existing systems (Ring, Nest) revealed they are often expensive and lack granular user control.
- **User Pain Points:** identified that "always-on" cameras cause privacy concerns and false alarms create "alert fatigue".
- **Technological Constraint:** The solution had to run on low-cost hardware (Raspberry Pi 4).

1.2 Requirement Analysis (Collection Analysis)

The collected data was analyzed to filter feasible features for Version 1.0.

- **Feasibility Study:** It was determined that running deep learning continuously on a Pi is impossible. **Decision:** Use a trigger-based system (PIR Sensor -> Camera -> AI) to save resources.
- **Conflict Resolution:** High-definition video requires massive storage. **Decision:** Implement "Intrusion Mode" logic—recording only 1 minute of video initially, then switching to interval images to save bandwidth.
- **Latency Analysis:** Standard HTTP requests are too slow for security alerts. **Decision:** Use Socket.IO for real-time bi-directional communication.

1.3 Requirement Specification

This document serves as the formal specification, detailing the functional and non-functional requirements derived from the analysis.

1.4 Requirement Validation

Validation will be performed through the development of the prototype. Success is defined by the system's ability to successfully filter a non-human motion event (false positive) versus a human motion event (true positive).

2. The Purpose of the Project

2.1 The User Business / Problem

Homeowners currently face a trade-off between security and privacy. Existing systems are either "Non-intelligent" (recording everything, wasting storage) or "cloud-dependent" (risking privacy). Users need a system that intelligently filters false alarms (pets, weather) and respects privacy when they are home.

2.2 Goals of the Project

- **Minimize False Alarms:** Integrate a Deep Learning model (SSD MobileNet V2) to filter motion events.
- **User Control:** Provide a seamless "Active/Inactive" mode toggle via a web dashboard.
- **Efficient Evidence Capture:** Automate the recording of a 1-minute video followed by 30-second interval stills during intrusions.
- **Real-Time Responsiveness:** Ensure alerts are received instantly via a Node.js server.

3. The Stakeholders

- **The Client/User:** Homeowners or small business owners seeking a custom security solution.
- **The Developer:** The dev team, responsible for full-stack implementation.
- **The Hardware:** The Raspberry Pi 4 unit acting as the edge computing device.
- **Faculty/Evaluators:** Academic supervisors reviewing the Final Year Project.

4. Mandated Constraints

- **Hardware:** The system must run on a Raspberry Pi 4 (4GB) with a Pi Camera Module V2 and HC-SR501 PIR sensor.
- **Connectivity:** The device must operate over Wi-Fi, utilizing Socket.IO for communication.
- **Development Stack:**
 - Edge: Python with TensorFlow Lite.
 - Backend: Node.js.
 - Database: MongoDB with GridFS.
 - Frontend: HTML5, CSS3, JavaScript.

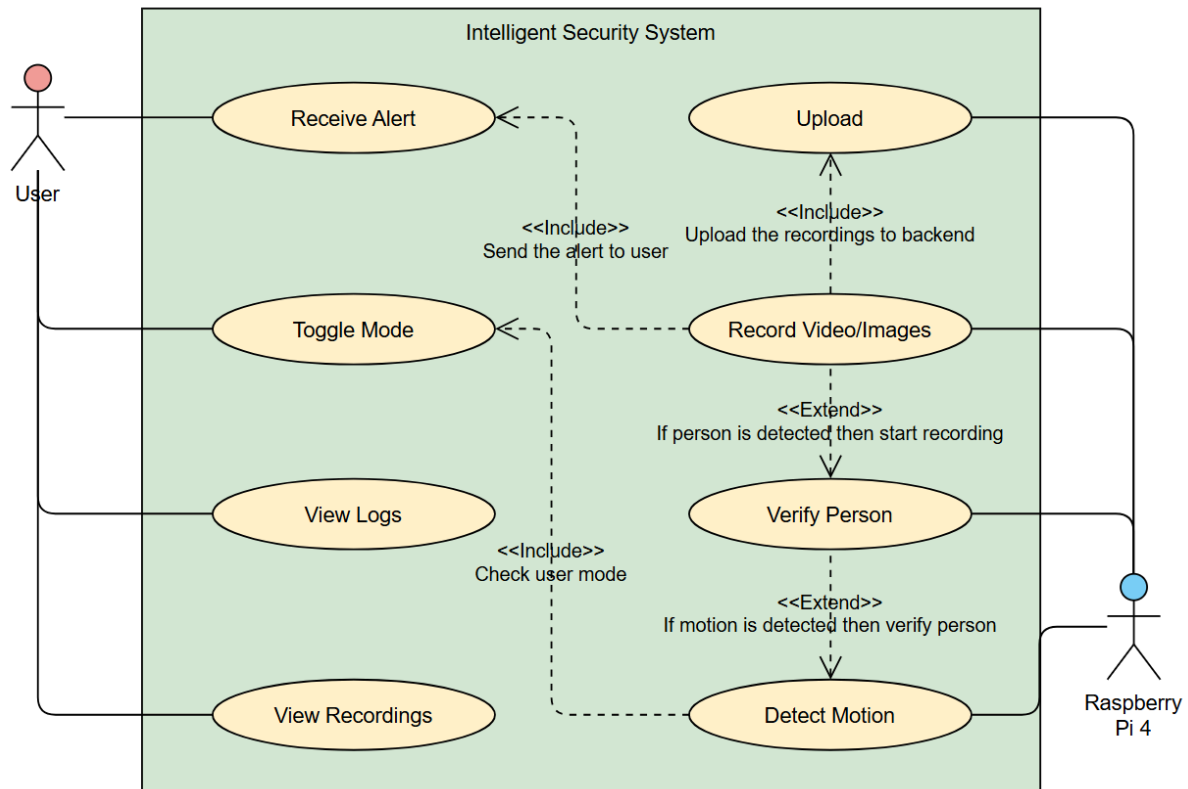
5. Scope of the Work (Use Case Diagram)

The following describes the Version 1.0 Use Case Diagram.

5.1 Diagram Description:

- **Actor: User**
 - **Toggle System Mode (Active/Inactive).**
 - **View Event Logs.**
 - **View Recorded Media (Videos/Images).**
 - **Receive Intrusion Alert.**
- **Actor: System (Raspberry Pi)**
 - **Detect Motion.**
 - **Verify Person (AI).**
 - **Record Video/Capture Still Image.**
 - **Upload Data.**
 - **Send Alert.**

5.2 Diagram:



6. Functional Requirements

6.1 Mode Management

- **FR-01:** The system shall allow the user to toggle between "Active" and "Inactive" modes via the web interface.
- **FR-02:** In "Inactive Mode," the system shall disable the camera and PIR sensor processing to ensure privacy.
- **FR-03:** In "Active Mode," the PIR sensor shall be armed to detect motion.

6.2 Motion Detection & AI Verification

- **FR-04:** Upon motion detection, the system shall capture a temporary frame and pass it to the TensorFlow Lite model.
- **FR-05:** The system shall determine if the detected object is a "Person" with a confidence threshold > 50%.
- **FR-06:** If a person is detected, the system shall enter "Intrusion Mode".
- **FR-07:** If no person is detected, the system shall discard the event and return to monitoring.

6.3 Intrusion Handling & Recording

- **FR-08:** Upon entering Intrusion Mode, the system shall immediately record a video of 60 seconds duration.
- **FR-09:** Following the video, the system shall capture high-resolution still images every 30 seconds as long as motion persists.
- **FR-10:** All media files shall be encoded and uploaded to the MongoDB GridFS storage.

6.4 Alerts & Communication

- **FR-11:** The system shall emit a security_alert event via Socket.IO immediately upon confirming a human threat.
- **FR-12:** The web dashboard shall display a visual alert (e.g., pop-up or color change) upon receiving the signal.

7. Non-Functional Requirements

7.1 Usability

- **NFR-01:** The web dashboard must be responsive (mobile-friendly) to allow access from smartphones.
- **NFR-02:** The mode toggle switch must provide visual feedback (e.g., Green for Active, Grey for Inactive) within 1 second.

7.2 Performance

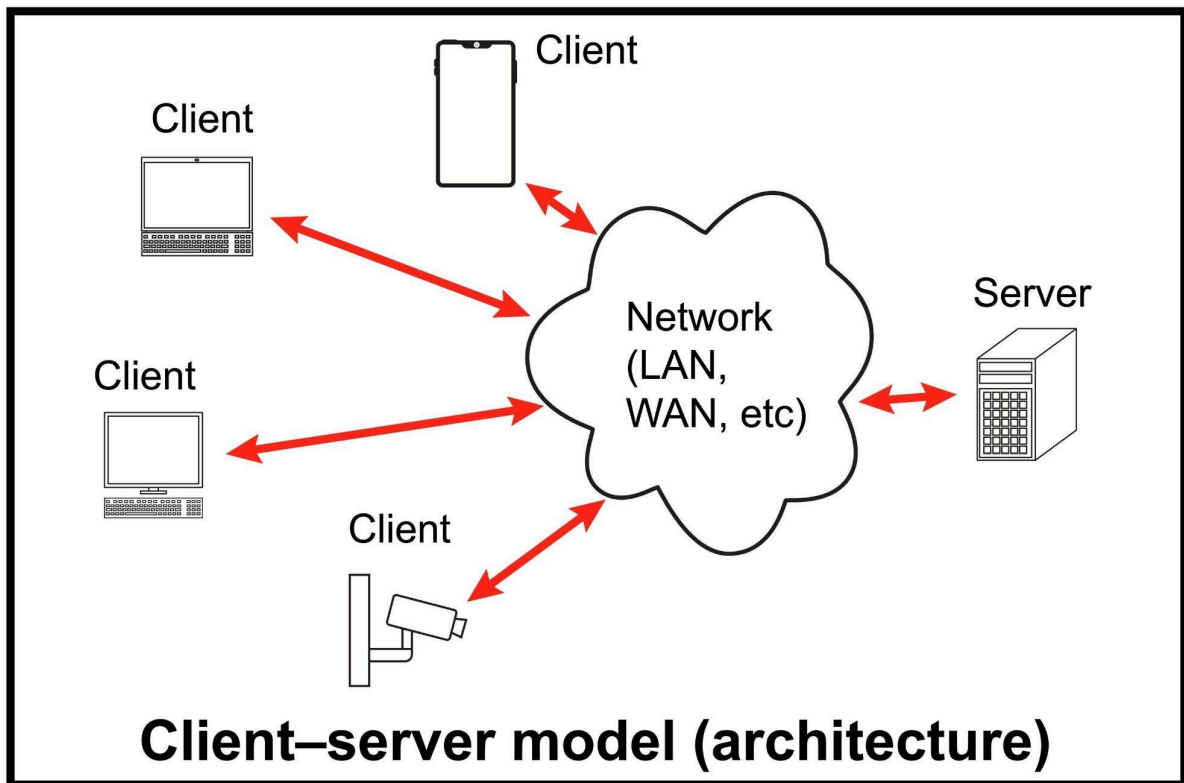
- **NFR-03:** The AI inference (person detection) must complete within 200ms on the Raspberry Pi 4.
- **NFR-04:** The latency between a detected intrusion and the user alert on the dashboard shall not exceed 2 seconds.

7.3 Reliability

- **NFR-05:** The system must automatically reconnect to the Socket.IO server if the Wi-Fi connection drops.
- **NFR-06:** False alarms (triggered by non-humans) should be reduced by at least 80% compared to standard PIR-only systems.

8. Architectural Design

This section details the system's components and their interactions.



8.1 High-Level Architecture

The system follows a Client-Server-Edge architecture.

1. The Edge (Raspberry Pi):

- Runs the main.py controller script.
- Manages hardware drivers (Camera, GPIO).
- Hosts the TFLite Interpreter.

2. The Server (Node.js):

- Acts as the central communication hub.
- Maintains the state (is_active boolean).
- Exposes API endpoints for the Frontend.

3. The Database (MongoDB):

- Collection events: Stores metadata (timestamp, threat_type, media_ids).
- Bucket fs (GridFS): Stores binary chunks of video/images.

4. The Client (Frontend):

- Single Page Application (SPA).
- Maintains a persistent WebSocket connection.

8.2 Data Flow Diagram (DFD) Description

1. User Input: User clicks "Activate" on Dashboard -> Socket emits toggle_active -> Server updates State -> Server emits state_change to Pi.

2. Sensor Input: PIR triggers -> Pi Camera captures frame -> TFLite Model infers Class.

3. Alert Flow: Pi confirms 'Person' -> Pi emits intrusion_detected to Server -> Server broadcasts to Dashboard.

4. Data Flow: Pi records video -> Pi sends HTTP POST with file stream -> Server receives stream -> Server pipes stream to GridFS.

9. Future Scope (Project Evolution)

- **v1.1:** Customizable intrusion timers.
- **v2.0:** Object detection for cars/packages.
- **v2.1:** Integration of environmental sensors (temperature/humidity).

Appendix: A Glossary of Terms & Acronyms

A

Active Mode

The state in which the security system is fully armed. In this mode, the PIR sensor is energized, and the camera is ready to verify motion events using the AI model.

API (Application Programming Interface)

A set of protocols and tools that allows the Raspberry Pi to communicate with the Node.js server. In this project, the API handles requests to upload video files and update device status.

C

COCO Dataset (Common Objects in Context)

A large-scale object detection, segmentation, and captioning dataset. The pre-trained model used in this project (SSD MobileNet) was trained on this dataset to recognize "persons" among 80+ other categories.

Confidence Threshold

A numerical value (0 to 1, or 0% to 100%) representing the machine learning model's certainty. For this project, a threshold (e.g., 50%) is set; if the model is less than 50% sure an object is a person, the system ignores it.

F

False Positive

An error in data reporting in which a test result improperly indicates the presence of a condition. In this project, a "False Positive" refers to the system raising a security alarm for a non-threat, such as a pet, shadow, or moving vehicle.

G

GridFS

A specification for storing and retrieving files that exceed the BSON-document size limit of 16 MB. This project uses GridFS within MongoDB to store large video files and high-resolution images generated during intrusions.

GPIO (General Purpose Input/Output)

Pins on the Raspberry Pi board that allow it to communicate with external hardware. The PIR motion sensor is connected to these pins to send electrical signals to the Python script when motion is detected.

I

Inactive Mode

The "privacy" state of the system. In this mode, the system logic ignores sensor inputs and disables the camera to ensure no recording occurs while the user is home.

Inference

The process where the pre-trained Machine Learning model processes live data (an image) to make a prediction (classify objects). This occurs locally on the Raspberry Pi.

Intrusion Mode

A specific system state triggered only when the AI model confirms a "Person." This mode executes a defined protocol: recording a 1-minute video immediately, followed by capturing still images every 30 seconds.

J

JSON (JavaScript Object Notation)

A lightweight data-interchange format. It is used to structure the messages sent between the Raspberry Pi and the Server (e.g., sending an alert object: { "type": "alert", "confidence": 0.85 }).

L

Latency

The time delay between a cause and an effect. In this project, low latency is critical to ensure that the user receives a security alert on their phone immediately after the intruder is detected.

M

MongoDB

A NoSQL database program. It is used in this project to store event logs (metadata) and large media files (via GridFS).

N

Node.js

A JavaScript runtime built on Chrome's V8 engine. It acts as the backend server, handling API requests and managing the real-time Socket.IO connections.

P

PIR Sensor (Passive Infrared Sensor)

An electronic sensor that measures infrared (IR) light radiating from objects in its field of view. It is used as the primary "wake-up" trigger for the system to detect initial motion.

R

Raspberry Pi 4

A low-cost, credit-card-sized single-board computer. It serves as the "Edge" device in this project, handling the camera, sensor, and AI processing.

S

Socket.IO

A library that enables real-time, bi-directional, and event-based communication between the web client and the server. It is chosen over HTTP to ensure instant alert delivery.

SPA (Single Page Application)

A web application (Frontend) that interacts with the user by dynamically rewriting the current web page with new data from the web server, rather than loading entire new pages.

SSD MobileNet V2

A Convolutional Neural Network (CNN) architecture. "SSD" (Single Shot Detector) refers to the method of detecting objects, and "MobileNet" refers to the lightweight internal structure optimized for mobile/embedded devices like the Raspberry Pi.

State-Aware Logic

The core proprietary logic of this project where the system changes its behavior (recording vs. ignoring) based on a user-defined context (Active vs. Inactive) rather than operating in a static loop.

T

TensorFlow Lite (TFLite)

An open-source deep learning framework for on-device inference. It is used to run the "heavy" object detection model efficiently on the Raspberry Pi's limited hardware.